

SAGRISA

Fuente Técnica Maestra de Arquitectura, Integración y Gobierno de la Plataforma Multicanal

Documento técnico de referencia

Versión base consolidada a partir de las fuentes funcionales y estratégicas del proyecto

Cliente	SAGRISA S.A. de C.V.
Proyecto	Plataforma Multicanal / Modernización técnica sobre entorno Microsoft
Propósito del documento	Consolidar una fuente técnica madre para servir como base de diseño, implementación, estimación, desarrollo y futuras propuestas técnicas.
Arquitectura objetivo	React PWA + Azure API Management + ASP.NET Core APIs + Mirror SQL de consulta + Dynamics 365 como sistema fuente + Microsoft Entra ID + OpenAPI/Swagger + Application Insights.

Nota técnica. Este documento toma como antecedente la Ruta Digital, el Plan de Implementación y la matriz de roles existente; sin embargo, normaliza y reemplaza varias premisas preliminares para alinearse con la decisión final adoptada: frontend React PWA, servicios ASP.NET Core, consumo vía APIM, autenticación con Entra ID, documentación OpenAPI/Swagger, observabilidad con Application Insights y uso de mirror SQL como capa de consulta.

Contenido

1. 1. Naturaleza y objetivo de la fuente técnica
2. 2. Fuentes base consideradas y criterio de consolidación
3. 3. Contexto actual de SAGRISA y premisas de diseño
4. 4. Decisiones de arquitectura adoptadas
5. 5. Arquitectura objetivo por capas
6. 6. Capa de experiencia: React PWA
7. 7. Capa de exposición y gobierno: Azure API Management
8. 8. Capa de servicios: ASP.NET Core Web APIs
9. 9. Capa de datos e integración: Mirror SQL y Dynamics 365
10. 10. Seguridad, identidad y autorización
11. 11. Contrato API, OpenAPI/Swagger y lineamientos de versionado
12. 12. Monitoreo, trazabilidad y observabilidad con Application Insights
13. 13. Dominios funcionales del sistema
14. 14. Roles, permisos y modelo de acceso
15. 15. Reglas transversales de negocio, lectura y escritura
16. 16. Lineamientos de integración con Dynamics 365
17. 17. Ambientes, calidad técnica y ruta de implementación
18. 18. Riesgos técnicos y mitigaciones
19. 19. Criterios de aceptación técnica
20. 20. Fuentes base y premisas sustituidas

1. Naturaleza y objetivo de la fuente técnica

Esta fuente técnica maestra tiene como objetivo consolidar, ordenar y explicar de forma amplia la solución tecnológica que SAGRISA adoptará para modernizar su plataforma actual dentro de un entorno plenamente alineado con Microsoft, pero bajo una lógica de costo razonable, control arquitectónico y desacople funcional.

Su propósito no es repetir el discurso comercial de las rutas previas, sino transformar ese material en un documento técnico de referencia que sirva para cinco fines concretos: primero, documentar la arquitectura objetivo; segundo, establecer criterios claros para diseño y desarrollo; tercero, dejar trazadas las reglas de integración y seguridad; cuarto, servir como base de futuras propuestas técnicas o documentos de alcance; y quinto, convertirse en la nueva fuente madre desde la cual puedan derivarse anexos, cronogramas, matrices, diagramas o TDR internos.

En consecuencia, este documento asume una postura deliberadamente técnica. Explica qué componentes componen la solución, por qué se eligieron, qué problema resuelven, cómo se relacionan entre sí, qué límites deben respetar y cuáles son las reglas que permitirán que la plataforma evolucione sin caer nuevamente en dependencias frágiles o en una arquitectura difícil de sostener.

2. Fuentes base consideradas y criterio de consolidación

La consolidación aquí presentada parte de tres grupos de insumos. El primero está conformado por la Ruta Digital SAGRISA 2026, donde se describe la evolución del negocio hacia una experiencia multicanal, conectada y en tiempo real, así como el rol central del entorno Dynamics Cloud dentro de la transformación. El segundo corresponde al Plan de Implementación de Fase 1, que aterriza componentes, etapas y actores

del proyecto. El tercero reúne las fuentes funcionales de roles y pantallas, incluyendo la matriz consolidada de roles y los artes o pantallas de cliente, director, gerente, supervisor y vendedor. Estas fuentes describen el comportamiento esperado por perfil, la operación cotidiana y las necesidades de consulta, pedido, cobro, facturación, metas, inventario y autorización. Fuente base analizada: RUTA-DIGITAL.pdf, RUTA-DIGITAL 2026 Version 2 (1).pdf, PLAN-DE-IMPLEMENTACION (2).pdf y SAGRISA ROLES (1).docx.

Fuente base funcional y estratégica consolidada a partir de RUTA-DIGITAL / PLAN-DE-IMPLEMENTACION / SAGRISA ROLES.

El criterio de consolidación aplicado no fue copiar cada premisa textual de los documentos existentes, sino separar dos capas de información. Por un lado, se conservó aquello que sigue siendo válido como visión de negocio: la experiencia multicanal, la conexión con Dynamics, la orientación a tiempo real, la analítica, la trazabilidad y el soporte al usuario interno y externo. Por otro lado, se sustituyeron algunas decisiones de implementación preliminares que ya no representan la arquitectura final acordada.

En términos prácticos, las fuentes anteriores hablaban de una app React Native, de un middleware con N8N y del uso de Azure Active Directory con nomenclatura antigua. La arquitectura final acordada para esta fuente técnica cambia ese planteamiento a una solución centrada en React PWA, Azure API Management como gateway, ASP.NET Core como capa de servicios, Microsoft Entra ID como proveedor de identidad, OpenAPI/Swagger para contrato técnico y Application Insights como capa de observabilidad. Asimismo, se incorpora como premisa central el uso de una base espejo SQL para consultas de alta demanda, mientras Dynamics y el sistema fuente siguen concentrando la verdad transaccional.

Este documento, por tanto, no invalida las fuentes previas como antecedentes; las reordena y las depura para dejar una versión técnica más coherente, ejecutable y sostenible.

3. Contexto actual de SAGRISA y premisas de diseño

SAGRISA ya cuenta con una trayectoria tecnológica relevante. Las fuentes previas describen que la organización desarrolló hace aproximadamente una década una aplicación móvil que se mantuvo operativa y útil durante años, y que posteriormente migró todo su entorno Dynamics a la nube, dejando a la aplicación original todavía activa en un esquema separado y no totalmente alineado al nuevo entorno cloud. Ese antecedente es importante porque demuestra dos cosas al mismo tiempo: primero, que la empresa ha sabido invertir en tecnología con visión; y segundo, que el punto actual exige una actualización estructural para no depender de componentes legados, integraciones rígidas o un canal móvil aislado de la arquitectura principal.

El proyecto, además, no debe interpretarse como una simple renovación visual. Las fuentes existentes muestran que la operación deseada abarca consulta de productos y precios, pedidos, cobros, facturas, metas comerciales, autorizaciones jerárquicas, visibilidad regional y soporte interno. Es decir, la nueva plataforma debe responder tanto a una realidad transaccional y operativa como a una realidad analítica y de control.

De ese contexto surgen las premisas de diseño que gobiernan esta fuente técnica:

- No exponer directamente Dynamics al frontend ni convertir al ERP en una API pública improvisada.
- Crear una capa propia de servicios para desacoplar la experiencia de usuario respecto al sistema fuente.
- Separar consultas de alta demanda de operaciones transaccionales sensibles, utilizando un mirror SQL solo para lectura.
- Centralizar el acceso a través de un gateway de APIs que permita reglas, seguridad, trazabilidad y monitoreo.

- Usar un frontend unificado que reduzca costos de desarrollo y mantenimiento sin perder experiencia multiplataforma.
- Construir una base que pueda crecer luego hacia atención al cliente, IA interna, analítica y otros componentes sin rehacer la solución completa.

4. Decisiones de arquitectura adoptadas

La arquitectura final adoptada para SAGRISA obedece a un criterio de equilibrio entre alineación corporativa, sostenibilidad operativa y control de costos. No se trata de usar cada producto posible del ecosistema Microsoft, sino de usar los que realmente aportan orden estructural al proyecto.

Componente	Decisión adoptada	Justificación técnica
Frontend	React PWA	Permite una sola base de código, experiencia web moderna, despliegue rápido y operación multiplataforma sin el costo de una app nativa independiente.
Gateway	Azure API Management	Centraliza exposición de APIs, validación de tokens, políticas, límites, versionado, analítica y control de entrada.
Servicios	ASP.NET Core Web APIs	Alinea la solución con Microsoft en backend, ofrece control total del dominio de negocio y permite construir endpoints propios desacoplados.
Identidad	Microsoft Entra ID	Resuelve autenticación corporativa, SSO, emisión de tokens, integración con políticas de acceso y un modelo estándar de seguridad.
Consultas	Mirror SQL / Base espejo	Descarga la presión de lectura del sistema fuente y acelera búsquedas, catálogos, históricos e indicadores consultivos.
Transacción	Dynamics 365 + sistemas fuente autorizados	Mantiene una única verdad operativa para escritura, autorizaciones, actualización de estados y operaciones críticas.
Contrato API	OpenAPI + Swagger UI	Formaliza la documentación técnica, facilita pruebas, revisión funcional y consumo ordenado

		por parte de APIM y equipos técnicos.
Observabilidad	Application Insights	Permite monitoreo centralizado de disponibilidad, rendimiento, errores, latencia y trazas operativas.

Estas decisiones tienen un efecto importante sobre el proyecto: convierten la plataforma en un sistema con contratos definidos, capas distinguibles y responsabilidades claras. El frontend deja de cargar con la lógica; el gateway deja de ser una simple publicación; la API se vuelve la pieza donde vive el negocio; el mirror deja de ser una tentación de escritura y se convierte en un acelerador de lectura; y Dynamics recupera su papel correcto como núcleo de operación y fuente autorizada.

5. Arquitectura objetivo por capas

SAGRISA | Arquitectura Técnica Objetivo

React PWA + APIM + ASP.NET Core + Mirror SQL + Dynamics 365

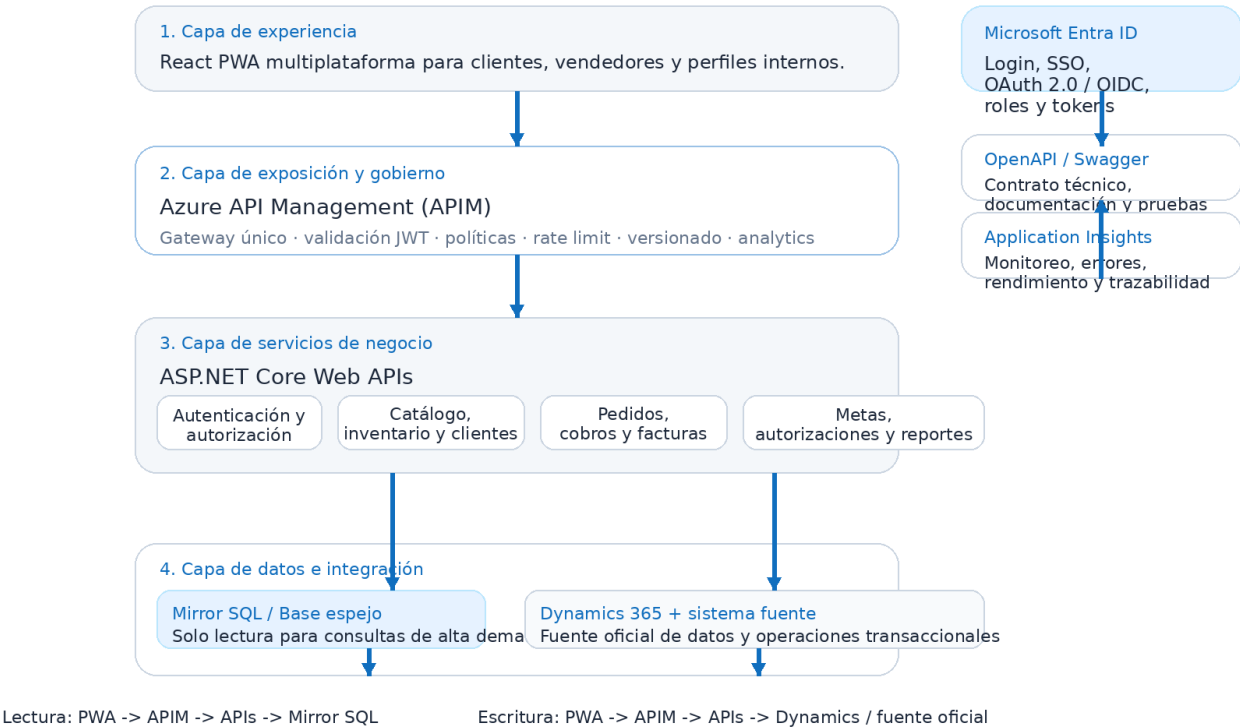


Figura 1. Arquitectura técnica objetivo consolidada para SAGRISA.

La arquitectura objetivo se compone de cuatro capas principales y dos capacidades transversales. Las capas principales son experiencia, exposición y gobierno, servicios de negocio, y datos e integración. Las capacidades transversales son identidad/seguridad y observabilidad/documentación. Esta organización es esencial porque impide que la solución crezca como un conjunto de conexiones puntuales y, en cambio, la obliga a evolucionar como una plataforma.

La capa de experiencia concentra todo lo que los usuarios ven y operan. La capa de exposición y gobierno se encarga de canalizar las solicitudes, validar acceso y aplicar políticas comunes. La capa de servicios interpreta el negocio y ejecuta reglas. La capa de datos e integración consume información del mirror y de los sistemas fuente. Sobre todas ellas operan Microsoft Entra ID para autenticación y Application Insights/OpenAPI para monitoreo y control técnico.

Nota técnica. El principio estructural más importante es el siguiente: ningún cliente, portal o PWA debe hablar directamente ni con Dynamics ni con la base de datos espejo. Todo consumo debe pasar por la capa API y el gateway.

6. Capa de experiencia: React PWA

6.1 Rol de la PWA dentro del ecosistema

La PWA será la puerta principal de interacción para los perfiles comerciales, operativos y de consulta. Esta decisión sustituye la idea preliminar de una app móvil separada y de un portal distinto, concentrando la experiencia en una sola capa tecnológica que podrá operar desde navegador de escritorio, tablet o dispositivo móvil con comportamiento similar al de una aplicación instalable.

La ventaja de esta decisión no es únicamente presupuestaria. Desde el punto de vista técnico, la PWA reduce duplicidad de lógica de presentación, facilita ciclos de publicación más rápidos, mejora la mantenibilidad y permite que toda la conversación entre interfaz y backend se apoye en contratos HTTP bien definidos.

6.2 Alcances funcionales de la PWA

La información funcional proveniente de las fuentes de roles permite identificar que la PWA deberá cubrir, al menos, cinco grandes experiencias de usuario: experiencia de consulta de cuenta para cliente; experiencia comercial para vendedor; experiencia de seguimiento operativo para supervisor; experiencia de control y autorización para gerente; y experiencia ejecutiva de indicadores para director.

Perfil	Experiencias principales en PWA	Observación técnica
Cliente	Estado de cuenta, facturas, historial, antigüedad de saldo, detalle de pedido, configuración de acceso	Predomina la lectura y autoservicio.
Vendedor	Clientes, inventario, pedidos, cobros, historial, metas personales	Requiere interacción constante y tiempos de respuesta ágiles.
Supervisor	Búsqueda de clientes, seguimiento de pedidos, cobros, facturas, metas grupales, control operativo	Comparte parte del flujo con vendedor pero con mayor alcance.
Gerente	Metas, inventario, autorizaciones, revisión de facturas, validaciones	Necesita autorización formal y trazabilidad de decisiones.

Director	Metas regionales, ventas y cobros por país/área, inventario consolidado, visión ejecutiva	Debe mantenerse como perfil de lectura estratégica.
----------	---	---

6.3 Principios de diseño de la capa de experiencia

La PWA debe diseñarse bajo tres criterios. El primero es desacople: ninguna vista debe depender de una estructura interna de Dynamics. El segundo es foco por rol: cada perfil debe ver solo aquello que necesita para operar o decidir. El tercero es rendimiento percibido: el usuario debe sentir continuidad, incluso cuando una operación requiera validación o consulta en varias fuentes.

A nivel técnico, esto implica usar un esquema claro de módulos, navegación basada en permisos, manejo consistente de errores, consumo controlado por API y estados visibles para operaciones sensibles, por ejemplo cuando un pedido cambia de pendiente a problema de crédito o cuando un cobro requiere comprobante, firma o selección de método de pago.

7. Capa de exposición y gobierno: Azure API Management

Azure API Management será la puerta única de entrada hacia los servicios del proyecto. Esta pieza no se introduce solo para 'publicar APIs', sino para ordenar el gobierno técnico del ecosistema. En un proyecto como SAGRISA, donde coexistirán perfiles internos, operaciones críticas, contratos múltiples y posiblemente futuras integraciones externas, la falta de un gateway central terminaría generando APIs expuestas de forma desigual, reglas duplicadas y una seguridad difícil de sostener.

7.1 Responsabilidades concretas de APIM

- Recibir todas las llamadas originadas por la PWA o por futuros consumidores autorizados.
- Validar tokens emitidos por Microsoft Entra ID antes de permitir el paso al backend.
- Aplicar políticas comunes como rate limiting, validación de headers, transformación simple de solicitudes o respuestas y control de versiones.
- Concentrar analítica básica de consumo y facilitar trazabilidad sobre qué endpoint se llamó, con qué resultado y en qué volumen.
- Convertirse en la fachada técnica estable del proyecto, aunque los servicios internos evolucionen.

7.2 Qué sí hace APIM y qué no debe hacer

APIM sí debe gobernar la exposición, pero no debe reemplazar la lógica de negocio. La tentación típica en este tipo de proyectos es empezar a resolver reglas funcionales directamente en el gateway. Esa práctica puede parecer rápida al inicio, pero termina fragmentando el comportamiento del sistema. En esta fuente técnica se deja establecido que las validaciones generales y las políticas transversales viven en APIM; las reglas del negocio viven en ASP.NET Core.

Ejemplos de reglas aptas para APIM son validación de JWT, límites por cliente, validación de cabeceras y enrutamiento por versión. Ejemplos de reglas que deben quedar en la API son cálculo de disponibilidad según bodega, autorización de descuento, validación de crédito, consolidación de cobro múltiple o interpretación de estados de pedido.

8. Capa de servicios: ASP.NET Core Web APIs

La capa de servicios es el corazón real de la solución. Aquí vive el negocio. Aquí se transforma una necesidad funcional de pantalla en un contrato, una regla, una validación, una consulta consolidada o una transacción autorizada. Esta capa es la que permite que la modernización de SAGRISA no dependa del modelo interno de Dynamics, sino de una fachada de servicios propia, controlada y expresiva.

8.1 Organización por dominios

Dominio API	Responsabilidad	Ejemplos de capacidades
Identity / Session	Intermediar claims, perfiles, sesión y datos de inicio	Perfil activo, módulos visibles, datos básicos del usuario y permisos efectivos
Customers	Consulta de clientes, cuenta, crédito y datos asociados	Búsqueda de clientes, cuenta, saldo, crédito disponible, antigüedad
Catalog	Productos, bodegas, lotes, precios, costos y disponibilidad	Inventario por bodega, precio base, costo, familias, filtros
Orders	Creación, edición, confirmación y seguimiento de pedidos	Pedidos abiertos, histórico, detalle, observaciones, fechas y direcciones
Collections	Cobros, métodos de pago, historial y trazabilidad	Anticipo, pago parcial, pago total, pago múltiple, comprobante y firma
Invoices	Facturas, detalle, estados y autorizaciones relacionadas	Consulta por cliente, detalle de factura, estados, saldos
Goals & Analytics	Metas, ventas, cobros y tableros por periodo y rol	Mensual, trimestral, anual, por vendedor, por país y por área
Approvals	Flujos jerárquicos y decisiones formales	Autorización de facturas, denegación, anulación controlada, observaciones

8.2 Reglas de construcción de la capa API

- Cada endpoint debe responder a un caso de uso concreto y no a una simple tabla del sistema fuente.
- Las APIs deben ser expresivas para el frontend; no meros proxys del ERP.
- Las respuestas deben devolver estructuras ya útiles para la PWA, evitando que el frontend tenga que recomponer reglas de negocio.
- Las APIs deben dejar trazabilidad de operaciones críticas, especialmente en pedidos, cobros, autorizaciones y cambios de precio.
- Las operaciones deben distinguir claramente lectura, prevalidación y confirmación transaccional.

Esto es importante porque muchas arquitecturas caen en el error de construir servicios demasiado crudos, donde el frontend termina rearmando significado. En SAGRISA, la API debe ser la traductora formal entre el negocio y la interfaz.

9. Capa de datos e integración: Mirror SQL y Dynamics 365

La capa de datos de esta solución no es una sola cosa. Se compone, en realidad, de dos planos distintos: un plano de consulta y un plano de verdad transaccional. El plano de consulta lo ocupa la base espejo o mirror SQL que SAGRISA proveerá para atender lecturas. El plano de verdad transaccional lo mantienen Dynamics 365 y los sistemas fuente autorizados.

9.1 Rol del mirror SQL

El mirror SQL existe para acelerar y estabilizar. Su propósito es ofrecer una superficie de lectura que permita consultar con comodidad catálogos, productos, existencias, históricos, facturas, saldos o agregados, sin castigar el núcleo transaccional. También facilita que el equipo de desarrollo construya endpoints de consulta ricos y rápidos, especialmente para experiencias de inventario, búsqueda y dashboard.

Sin embargo, esta fuente técnica deja una regla innegociable: el mirror es read-only desde la lógica de la aplicación. Bajo ninguna circunstancia debe convertirse en el lugar donde la PWA crea, modifica o confirma operaciones oficiales.

9.2 Rol de Dynamics y el sistema fuente

Dynamics 365 y los sistemas oficiales asociados conservan la autoridad sobre aquello que constituye la operación del negocio: generación y actualización oficial de transacciones, estados definitivos, autorizaciones, flujos que comprometen integridad y cualquier operación que deba reflejarse como verdad empresarial.

Tipo de operación	Origen recomendado	Destino autorizado	Criterio
Consulta de catálogo/inventario	PWA -> APIM -> API	Mirror SQL	Velocidad y alta frecuencia
Consulta de historial/cuenta	PWA -> APIM -> API	Mirror SQL	Experiencia consultiva
Creación/edición de pedido	PWA -> APIM -> API	Sistema fuente / Dynamics	Transacción oficial
Registro de cobro	PWA -> APIM -> API	Sistema fuente / Dynamics	Trazabilidad y consistencia
Autorización / denegación	PWA -> APIM -> API	Sistema fuente / Dynamics	Decisión jerárquica formal
Reportería agregada	PWA -> APIM -> API	Mirror SQL y/o servicios preparados	Consulta y resumen

Nota técnica. La separación lectura/escritura no es decorativa. Es la condición que evita doubles verdades, operaciones huérfanas y discusiones futuras sobre cuál base 'manda' realmente.

10. Seguridad, identidad y autorización

La seguridad de la solución se compone de tres niveles: autenticación, autorización técnica y autorización funcional. La autenticación la resuelve Microsoft Entra ID. La autorización técnica corresponde a la validación del token y de los claims mínimos para invocar APIs. La autorización funcional corresponde a las reglas de negocio que determinan qué puede hacer un usuario dentro del sistema según su perfil real.

10.1 Autenticación con Microsoft Entra ID

El login de la PWA se apoyará en Microsoft Entra ID mediante OAuth 2.0 y OpenID Connect. Esto significa que la identidad del usuario no se resolverá con una base local improvisada, sino con el proveedor corporativo, que emitirá los tokens necesarios para iniciar sesión y consumir APIs.

El resultado práctico es que SAGRISA obtiene inicio de sesión estandarizado, SSO, manejo ordenado de identidad y una base más segura para crecimiento futuro. La PWA se encargará de solicitar y portar el token; APIM validará el acceso inicial; y ASP.NET Core aplicará la autorización fina.

10.2 Autorización funcional y rol efectivo

Las fuentes de roles muestran una organización clara: cliente, director, gerente, supervisor y vendedor. Sin embargo, también muestran que el nombre del cargo no basta por sí solo para definir permisos. En varios casos existen solapamientos entre supervisor y vendedor, o entre niveles de consulta y validación. Por eso esta fuente técnica recomienda separar la capa organizacional de la capa de permisos del sistema.

Rol organizacional	Perfil funcional del sistema	Observación
Cliente	Cliente externo de consulta	Debe ver solo su información y administrar su acceso.
Vendedor	Usuario comercial operativo	Puede crear pedidos y registrar cobros dentro de su alcance.
Supervisor	Usuario supervisor comercial	Opera y supervisa, con mayor alcance sobre cartera/equipo.
Gerente	Usuario aprobador / táctico-operativo	Aprueba, deniega y controla procesos sensibles.
Director	Usuario ejecutivo de lectura estratégica	No ejecuta transacciones del día a día.

La separación anterior permitirá que el sistema crezca sin quedar amarrado al nombre del cargo. Es perfectamente posible que en el futuro exista un usuario gerente sin privilegio de reversa, o un supervisor con acceso ampliado a ciertos reportes. Por eso el rol efectivo debe quedar determinado por permisos explícitos y no únicamente por etiqueta organizacional.

11. Contrato API, OpenAPI/Swagger y lineamientos de versionado

Una plataforma de esta naturaleza no puede sostenerse a punta de endpoints opacos o de documentación oral. Cada API deberá describirse formalmente mediante OpenAPI, y esa definición será la base del contrato entre backend, frontend, pruebas, gateway y futuro mantenimiento.

Swagger UI se utilizará como interfaz de exploración y prueba del contrato OpenAPI. En términos sencillos, OpenAPI será el estándar que describe la API y Swagger UI la cara visible para leerla y probarla. Esto tiene un

valor técnico muy alto: elimina ambigüedad, facilita revisión de payloads, hace visible la seguridad aplicada a cada operación y ordena la conversación entre equipos.

11.1 Reglas mínimas del contrato

- Cada endpoint debe tener propósito, descripción y ejemplos claros.
- Los esquemas de request/response deben representar casos de uso, no solo tablas técnicas.
- Los códigos de respuesta deben estar documentados y ser consistentes.
- Las operaciones sensibles deben declarar claramente sus requisitos de seguridad y autorización.
- Las APIs deben versionarse desde el diseño para evitar rupturas futuras con la PWA.

La regla práctica será trabajar siempre bajo un modelo contrato primero o, como mínimo, contrato en paralelo al desarrollo. Ningún endpoint debería darse por finalizado sin un documento OpenAPI correspondiente.

12. Monitoreo, trazabilidad y observabilidad con Application Insights

Uno de los vacíos más costosos en muchos proyectos es que la solución se construye, se publica y luego, cuando algo falla, nadie sabe dónde mirar. Application Insights se incorpora precisamente para eliminar ese punto ciego. Esta capacidad permitirá conocer disponibilidad, tiempos de respuesta, errores, comportamiento por endpoint, trazas de ejecución y señales de degradación antes de que el problema escale.

Desde la perspectiva operativa, Application Insights debe registrar al menos cuatro dimensiones: desempeño de endpoints, errores y excepciones, trazas relevantes del negocio y pruebas de disponibilidad. Su objetivo no es almacenar ruido, sino generar visibilidad útil.

Dimensión	Qué debe observarse	Uso esperado
Disponibilidad	Endpoints críticos y salud del servicio	Detectar caídas o interrupciones
Rendimiento	Latencia por API, operación y dependencia	Detectar cuellos de botella
Errores	Excepciones, fallos de integración, respuestas 5xx y 4xx relevantes	Acelerar diagnóstico
Trazabilidad	Correlación entre solicitud, usuario, operación y dependencia	Entender el recorrido de una transacción

Esto es especialmente importante en SAGRISA porque la solución no será una pantalla aislada, sino una cadena completa que involucra identidad, gateway, APIs, mirror y sistema fuente. Si no se instrumenta desde el inicio, después será mucho más difícil reconstruir incidentes.

13. Dominios funcionales del sistema

La matriz de roles y pantallas permite identificar con claridad los dominios funcionales principales que la solución debe cubrir. Aunque la PWA los presentará como módulos o secciones, desde la arquitectura conviene verlos como dominios de negocio que luego se expresan en servicios, permisos y operaciones.

13.1 Dominio de cuenta y consulta financiera

Este dominio está fuertemente ligado al perfil cliente, pero también alimenta consultas parciales de otros usuarios. Incluye saldo adeudado, crédito disponible, estado de cuenta, historial de facturas, antigüedad de saldo y detalle de movimientos asociados.

13.2 Dominio de inventario y catálogo

Incluye productos, categorías, familias, bodegas, lotes, vencimientos, precios base y costos. Tiene alto valor transversal porque lo consumen vendedor, supervisor, gerente y director en distintos niveles.

13.3 Dominio de pedidos

Abarca búsqueda de clientes, selección de productos, modificación de cantidades, observaciones, fecha y dirección de entrega, consulta de pedidos abiertos, histórico y detalle. Es un dominio operativo, pero con efectos financieros y logísticos.

13.4 Dominio de cobros

Incluye anticipo, pago parcial, pago total y pago múltiple; selección de método de pago; captura de banco, referencia, cheque o transacción; comprobante; firma digital; DUI opcional y confirmación final de cobro.

13.5 Dominio de facturación y autorizaciones

Abarca consulta de facturas, detalle, revisión de estados y flujos donde un gerente aprueba, deniega o eventualmente participa en reversa controlada. Es uno de los dominios que más exige trazabilidad formal.

13.6 Dominio de metas y analítica

Comprende metas mensuales, trimestrales y anuales; ventas y cobros por área, por vendedor, por país y por periodo; indicadores de cumplimiento y vistas agregadas para supervisión y dirección.

14. Roles, permisos y modelo de acceso

Las fuentes revisadas permiten reconstruir un modelo bastante claro de acceso por rol. El cliente consulta; el vendedor ejecuta; el supervisor combina operación y seguimiento; el gerente controla y autoriza; y el director observa de forma estratégica. Esa lógica debe quedar expresada en el diseño de permisos, las APIs y la interfaz.

Rol	Enfoque	Acción predominante	Restricción estructural
Cliente	Consulta de cuenta	Ver su propia información	No altera procesos del negocio
Vendedor	Ejecución comercial	Crear pedidos y registrar cobros	No aprueba ni anula operaciones sensibles
Supervisor	Operación y seguimiento	Controlar ejecución del equipo	No debe concentrar autorizaciones formales mayores
Gerente	Control táctico	Aprobar, denegar y	Debe mantener

		validar	trazabilidad de sus decisiones
Director	Visión ejecutiva	Consultar agregados y cumplimiento	No opera transacciones diarias

Del análisis CRUD existente también se derivan reglas de diseño muy importantes: el cliente no altera datos operativos; el director no ejecuta transacciones; el gerente conserva la autorización formal; supervisor y vendedor comparten operación pero deben diferenciarse por límites; el cambio de precios debe quedar restringido; la anulación de cobros debe mantenerse supercontrolada; y todo cobro debe dejar evidencia completa de método, referencia, comprobante, firma y usuario ejecutor.

Nota técnica. Una futura evolución deseable es construir una matriz de permisos técnica desacoplada del cargo, de modo que el sistema administre privilegios por capacidad y no solo por nombre de rol.

15. Reglas transversales de negocio, lectura y escritura

Además de la separación de capas, esta fuente técnica deja definidas varias reglas transversales que deben gobernar el proyecto independientemente del módulo.

- Toda lectura de alta demanda debe priorizar el mirror SQL cuando la consistencia inmediata no sea crítica.
- Toda escritura oficial debe pasar por la API y terminar en el sistema fuente autorizado.
- Ningún cambio sensible debe ejecutarse solo desde la interfaz; siempre debe existir validación en backend.
- Las autorizaciones jerárquicas deben registrar usuario, fecha, operación, estado previo, decisión y justificación cuando aplique.
- Las operaciones de cobro deben ser rastreables de extremo a extremo, incluyendo adjuntos y firma.
- Las respuestas agregadas para metas y tableros deben salir preparadas desde la API, no armadas artesanalmente en la PWA.

16. Lineamientos de integración con Dynamics 365

La integración con Dynamics 365 debe concebirse como una relación formal entre sistemas, no como acceso directo de pantalla a ERP. El backend ASP.NET Core es quien se responsabiliza de traducir la necesidad funcional de la PWA a las interfaces de Dynamics y del sistema fuente. Esto permite encapsular complejidad, aislar cambios y evitar que el frontend se rompa cuando una integración de backend evolucione.

En esta arquitectura, Dynamics no es el frontend ni el gateway ni el contrato externo; es el sistema corporativo que participa como fuente oficial para operaciones. La API propia es la que decide cómo y cuándo interactuar con él, qué datos exponer, qué validar antes y qué responder después.

Cuando una operación requiera combinar mirror y fuente oficial, la API deberá hacerlo de manera explícita. Por ejemplo, puede consultar datos de apoyo desde el mirror, validar reglas propias y finalmente confirmar la transacción contra el sistema oficial. Lo importante es que la PWA siga viendo una sola fachada coherente.

17. Ambientes, calidad técnica y ruta de implementación

Aun cuando este documento no es un cronograma contractual, sí debe dejar lineamientos mínimos de ejecución. La propia documentación base ya planteaba una secuencia de análisis y arquitectura, desarrollo e

integración, pruebas y validación, y puesta en marcha. Esa lógica sigue siendo válida, pero aterrizada ahora a la arquitectura final adoptada.

Fase técnica	Objetivo	Resultado esperado
Arquitectura y contrato	Definir dominios, endpoints, permisos, integración y modelo de datos consultivo/transaccional	Documento técnico aprobado y mapa API inicial
Construcción base	Levantar APIM, APIs ASP.NET Core, PWA base, identidad y observabilidad	Esqueleto funcional integrado
Construcción funcional	Desarrollar módulos por dominio	Flujos operativos utilizables
QA y endurecimiento	Pruebas funcionales, seguridad, rendimiento y trazabilidad	Release Candidate con incidentes críticas resueltas
Salida a producción	Publicación controlada, validación y acompañamiento	Plataforma operativa bajo gobierno técnico

El proyecto debe contar al menos con ambientes de desarrollo, prueba/preproducción y producción. Asimismo, cada endpoint relevante debe llegar a QA con su contrato OpenAPI, sus pruebas mínimas y su telemetría conectada.

18. Riesgos técnicos y mitigaciones

Riesgo	Impacto	Mitigación propuesta
Confundir mirror con fuente oficial	Inconsistencias y dobles verdades	Regla estricta de read-only para mirror y escritura solo por sistema fuente.
Mover lógica a APIM	Fragmentación de reglas y mantenimiento difícil	Dejar APIM para gobierno transversal y backend para negocio.
Permisos basados solo en cargo	Autorizaciones ambiguas	Diseñar matriz de permisos efectiva separada del rol organizacional.
Falta de telemetría	Dificultad para diagnosticar incidentes	Instrumentar Application Insights desde fases tempranas.
Endpoints sin contrato formal	Ambigüedad entre equipos	Exigir OpenAPI/Swagger en todo endpoint relevante.
Frontend demasiado acoplado	Rupturas ante cambios internos	Consumir solo servicios propios y no estructuras directas de ERP.

19. Criterios de aceptación técnica

A nivel técnico, la solución deberá considerarse correctamente implantada cuando cumpla, como mínimo, con los siguientes criterios:

- La PWA opera por rol sin acceder directamente a sistemas fuente.
- Todas las llamadas pasan por APIM y respetan políticas de autenticación y gobierno.
- Los servicios ASP.NET Core encapsulan la lógica de negocio por dominio.
- El mirror SQL es utilizado solo en escenarios de lectura definidos.
- Las operaciones de escritura y autorización llegan al sistema fuente oficial.
- Los endpoints están documentados en OpenAPI y pueden probarse en Swagger UI.
- Application Insights captura disponibilidad, errores y rendimiento de forma útil.
- Los flujos de cobro, pedido, consulta y autorización dejan trazabilidad suficiente.

20. Fuentes base y premisas sustituidas

Fuentes base consideradas como antecedente funcional y estratégico:

- RUTA-DIGITAL.pdf / RUTA-DIGITAL 2026 Version 2 (1).pdf
- PLAN-DE-IMPLEMENTACION (2).pdf
- SAGRISA ROLES (1).docx
- Rol Cliente - 30novT.pdf
- Rol Director - 30novT (1).pdf
- Rol Gerente - 30novT.pdf
- Rol Supervisor - 30novT.pdf
- Rol Vendedor - 30novT.pdf

Premisas preliminares que esta fuente sustituye o normaliza:

- React Native como frontend principal, sustituido por React PWA.
- N8N como middleware central, sustituido por una capa propia de APIs ASP.NET Core con gobierno a través de APIM.
- Consumo ambiguo del ERP, sustituido por un modelo formal de fachada API y separación lectura/escritura.
- Nomenclatura antigua de Azure AD, normalizada a Microsoft Entra ID.
- Ausencia de una capa formal de contrato técnico, corregida mediante OpenAPI/Swagger.
- Ausencia de una línea explícita de observabilidad, cubierta mediante Application Insights.

Con ello, la presente fuente queda establecida como la referencia técnica consolidada para SAGRISA en esta etapa del proyecto. Cualquier documento técnico posterior —alcance detallado, propuesta económica, anexos funcionales, historias de usuario, matriz de endpoints, plan de pruebas o documento de ambientes— debería derivarse de esta base y no de las versiones previas por separado.

Nota técnica. Recomendación final: conservar este documento como fuente madre controlada, con versionado y fecha, para evitar que futuras conversaciones mezclen nuevamente visión comercial, decisiones preliminares y arquitectura técnica definitiva.

Apéndice A. Resumen de módulos y su correspondencia con roles

Como complemento rápido, se deja la siguiente tabla de correspondencia entre módulos visibles y perfiles principales, derivada de la matriz de roles existente.

Módulo	Cliente	Director	Gerente	Supervisor	Vendedor
Autenticación	Sí	Sí	Sí	Sí	Sí
Cuenta/ estado de cuenta	Sí	No	No	Parcial	Parcial
Facturas	Sí	Parcial	Sí	Sí	Parcial
Pedidos	Consulta	No	Sí	Sí	Sí
Inventario	Parcial	Sí	Sí	Sí	Sí
Metas	No	Sí	Sí	Sí	Sí
Cobros	Consulta	Parcial	Parcial	Sí	Sí
Autorizaciones	No	No visible	Sí	Parcial	No
Configuración	Sí	Sí	Sí	Sí	Sí